Проект «Эмулятор микрокалькулятора Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

Руководство по работе с терминалом

Версия инструкции: 15.10.2026

Терминал дает доступ к программной памяти, стеку и регистрам МК-61, внутреннему хранилищу, клавиатуре устройства, сервисным функциям и часам RTC. Настоящее руководство перенесено из Word-документа и приведено в соответствие с текущей прошивкой.

Подключение

Подключите MK61s mini к компьютеру по USB и откройте появившийся виртуальный последовательный порт в PuTTY, Tera Term, Arduino IDE Serial Monitor, `screen`, `minicom` или другой терминальной программе.

Рекомендуемые параметры:

- скорость 115200 бод;
- 8 бит данных, без контроля четности, 1 стоп-бит;
- без аппаратного и программного управления потоком;
- окончание строки CR, LF или CRLF;
- локальное эхо выключено: устройство само возвращает введенные символы;
- окно не меньше 112 столбцов и 40 строк для четырёх колонок `lasm`;
- кодировка Windows-1251.

При USB CDC значение скорости фактически не задает скорость USB-передачи, но 115200 соответствует настройке прошивки и подходит для всех терминальных программ. В режиме USB-диск последовательный терминал остановлен. Чтобы снова работать с командами, выйдите из режима USB-диска клавишей ESC на устройстве.

После подключения прошивка печатает версию и приглашение:

```
MK61s mini ver. Jul 19 2026(12:34:56)
/>
```

Перед > показывается текущий каталог C6. После запуска это корень `/`; после `cd /Programs` приглашение станет `/Programs>`. Если полный путь не помещается во внутренний буфер приглашения, вместо него показывается `...>`; pwd по-прежнему позволяет запросить путь явно.

Введите `help` и завершите строку клавишей Enter. Команды и английские имена регистров вводятся ASCII-символами. Имя команды пишется в нижнем регистре; специальная запись регистра начинается с заглавной R.

Кодировка терминала

Проводной протокол всегда использует Windows-1251; команды переключения кодировки больше нет. Внутренний текст устройства хранится в M8. Обычные байты M8 совпадают с CP1251, а стрелки и математические знаки, которых в ней нет, прошивка выводит понятными ASCII-последовательностями: `->`, `<-`, `!=`, `<=`, `>=`, `pi`, `sqrt`.

В Tera Term или PuTTY выберите Windows-1251. МКС и приложение USB Screen сами декодируют CP1251 в локальный Unicode и не меняют режим устройства. Поэтому F401 и F411 ведут себя одинаково, а русские имена и мнемоники больше не имеют отдельных исторических ветвей.

Редактор строки и история

Терминал принимает CR, LF и пары CRLF/LFCR. В ANSI-совместимых клиентах, включая PuTTY и Tera Term, стрелки влево и вправо перемещают курсор, `Backspace` стирает символ перед ним, а `Delete` — символ под ним. Ввод в середине строки работает в режиме вставки; `Home` и `End` переходят к началу и концу строки. Дополнительно поддерживаются стандартные сочетания `Ctrl+A`, `Ctrl+B`, `Ctrl+D`, `Ctrl+E` и `Ctrl+F`.

Стрелки вверх и вниз листают историю. Команда `history` печатает сохраненные строки. При возврате из истории восстанавливаются как незавершенная строка, так и положение курсора в ней.

Хранятся до 8 последних неповторяющихся подряд команд общим объемом до 256 байт. Максимальная полезная длина одной строки — 239 символов. Переполненная строка целиком отбрасывается и не исполняется.

Как устроена память МК-61

Программируемые калькуляторы семейства МК-61 имеют раздельные память программы и память данных. Программная память состоит из байтовых кодов операций, а стек и регистры памяти хранят числа в собственном десятичном формате калькулятора. Поэтому команды для программы, стека и регистров разделены.

Для программной памяти доступны как шестнадцатеричные коды, так и ассемблерные мнемоники. Условные переходы оригинального калькулятора выполняют переход при невыполнении написанного на клавише условия. Из-за этого названия некоторых ассемблерных переходов выглядят зеркально по отношению к надписям на клавишах.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	,	/ - /	ВП	Cx	В↑	F Bx
1	+	-	×	÷	↔	F 10^x	F e^x	F lg	F ln	F arcsin	F arccos	F arctg	F sin	F cos	F tg	
2	F π	F √‾	F x^2	F 1/x	F x^y	F ◯	К +	К -	К ×	К ÷	К ↔					
3	К 3	К 4	К 5	К 6	К 7	К 8	К 9	К ,	К / - /	К ВП	К Cx	К В↑				
4	П 0	П 1	П 2	П 3	П 4	П 5	П 6	П 7	П 8	П 9	П A	П B	П C	П Д	П ↑	
5	С/П	БП	В/О	ПП	К НОП	К 1	К 2	F X≠0	F L2	F X≥0	F L3	F L1	F X<0	F L0	F X=0	
6	ИП 0	ИП 1	ИП 2	ИП 3	ИП 4	ИП 5	ИП 6	ИП 7	ИП 8	ИП 9	ИП A	ИП B	ИП C	ИП Д	ИП ↑	
7	К X≠0 0	К X≠0 1	К X≠0 2	К X≠0 3	К X≠0 4	К X≠0 5	К X≠0 6	К X≠0 7	К X≠0 8	К X≠0 9	К X≠0 A	К X≠0 B	К X≠0 C	К X≠0 Д	К X≠0 ↑	
8	К БП 0	К БП 1	К БП 2	К БП 3	К БП 4	К БП 5	К БП 6	К БП 7	К БП 8	К БП 9	К БП A	К БП B	К БП C	К БП Д	К БП ↑	
9	К X≥0 0	К X≥0 1	К X≥0 2	К X≥0 3	К X≥0 4	К X≥0 5	К X≥0 6	К X≥0 7	К X≥0 8	К X≥0 9	К X≥0 A	К X≥0 B	К X≥0 C	К X≥0 Д	К X≥0 ↑	
A	К ПП 0	К ПП 1	К ПП 2	К ПП 3	К ПП 4	К ПП 5	К ПП 6	К ПП 7	К ПП 8	К ПП 9	К ПП A	К ПП B	К ПП C	К ПП Д	К ПП ↑	
B	К П 0	К П 1	К П 2	К П 3	К П 4	К П 5	К П 6	К П 7	К П 8	К П 9	К П A	К П B	К П C	К П Д	К П ↑	
C	К X<0 0	К X<0 1	К X<0 2	К X<0 3	К X<0 4	К X<0 5	К X<0 6	К X<0 7	К X<0 8	К X<0 9	К X<0 A	К X<0 B	К X<0 C	К X<0 Д	К X<0 ↑	
D	К ИП 0	К ИП 1	К ИП 2	К ИП 3	К ИП 4	К ИП 5	К ИП 6	К ИП 7	К ИП 8	К ИП 9	К ИП A	К ИП B	К ИП C	К ИП Д	К ИП ↑	
E	К X=0 0	К X=0 1	К X=0 2	К X=0 3	К X=0 4	К X=0 5	К X=0 6	К X=0 7	К X=0 8	К X=0 9	К X=0 A	К X=0 B	К X=0 C	К X=0 Д	К X=0 ↑	
F																

Классическая таблица кодов команд МК-61

По вертикали указана первая шестнадцатеричная цифра кода, по горизонтали — вторая.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	dot	neg	pow10	clr	push	preX
1	add	sub	mul	div	swap	e10	exp	lg	ln	asin	acos	atg	sin	cos	tg	?
2	pi	sqrt	sqr	rec	pow	rot	toM	?	?	?	toMS	?	?	?	?	?
3	inMS	mod	sgn	inM	int	frc	max	and	or	xor	not	rnd	?	?	?	?
4	sto0	sto1	sto2	sto3	sto4	sto5	sto6	sto7	sto8	sto9	stoA	stoB	stoC	stoD	stoE	?
5	hlt	jmp	ret	call	nop	?	?	jz	rpt2	jlz	rpt3	rpt1	jme	rpt0	jnz	?
6	ld0	ld1	ld2	ld3	ld4	ld5	ld6	ld7	ld8	ld9	ldA	ldB	ldC	ldD	ldE	?
7	jz[0]	jz[1]	jz[2]	jz[3]	jz[R4]	jz[5]	jz[6]	jz[7]	jz[8]	jz[9]	jz[A]	jz[B]	jz[C]	jz[D]	jz[E]	?
8	jmp[0]	jmp[1]	jmp[2]	jmp[3]	jmp[4]	jmp[5]	jmp[6]	jmp[7]	jmp[8]	jmp[9]	jmp[A]	jmp[B]	jmp[C]	jmp[D]	jmp[E]	?
9	jlz[0]	jlz[1]	jlz[2]	jlz[3]	jlz[4]	jlz[5]	jlz[6]	jlz[7]	jlz[8]	jlz[9]	jlz[A]	jlz[B]	jlz[C]	jlz[D]	jlz[E]	?
A	call[0]	call[1]	call[2]	call[3]	call[4]	call[5]	call[6]	call[7]	call[8]	call[9]	call[A]	call[B]	call[C]	call[D]	call[E]	?
B	sto[0]	sto[1]	sto[2]	sto[3]	sto[4]	sto[5]	sto[6]	sto[7]	sto[8]	sto[9]	sto[A]	sto[B]	sto[C]	sto[D]	sto[E]	?
C	jme[0]	jme[1]	jme[2]	jme[3]	jme[4]	jme[5]	jme[6]	jme[7]	jme[8]	jme[9]	jme[A]	jme0[B]	jme[C]	jme[D]	jme[E]	?
D	ld[0]	ld[1]	ld[2]	ld[3]	ld[4]	ld[5]	ld[6]	ld[7]	ld[8]	ld[9]	ld[A]	ld[B]	ld[C]	ld[D]	ld[E]	?
E	jnz[0]	jnz[1]	jnz[2]	jnz[3]	jnz[R4]	jnz[5]	jnz[6]	jnz[7]	jnz[8]	jnz[9]	jnz[A]	jnz[B]	jnz[C]	jnz[D]	jnz[E]	?
F	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

Таблица кодов с ассемблерными мнемониками

Клавиша калькулятора	Мнемоника	Смысл мнемоники
БП	JMP	безусловный переход, Jump
ПП	CALL	вызов подпрограммы, Call
В/О	RET	возврат, Return
С/П	HLT	останов, Halt
П→X	STO	запись, Store
X←П	LD	загрузка, Load
X≠0	JZ	переход при нуле
X≥0	JLZ	переход при отрицательном X
X<0	JME	переход при X больше или равном нулю
X=0	JNZ	переход при ненулевом X

Краткий справочник

Общие команды

Команда	Назначение
`ver`	Версия и дата сборки прошивки.
`identity`	Стабильная идентичность платы, сборки, USB и тома C6.
`date`	Чтение и установка даты и времени RTC.
`alarm`	Два аппаратных RTC-будильника A/B: разовый или ежедневный.
`help`	Актуальный список команд из самой прошивки.
`history`	История строк интерактивного терминала.

Программная память

Команда	Назначение
`list`	Вертикальная hex-таблица программной памяти.
`dump`	Последовательность hex-кодов программы.
`pub`	Листинг в журнальном формате.
`lasm`	Дизассемблированный листинг.
`isa`	Таблица ассемблерных мнемоник.
`asm`	Сборка мнемоник в программную память.
`ins`	Вставка одного opcode со сдвигом программы.
`hin`	Запись строки hex-байтов.
load <адрес> <путь.bin>	Распаковать бинарный образ в банки.
`hout`	Вывод программы в виде строк для `hin`.
`clr`	Очистка программной памяти с подтверждением.
`set$`	Совместимая служебная форма hex-записи.

Состояние калькулятора и клавиатура

Команда	Назначение
`reg`	Регистры памяти R0…RE.
`stk`	Стек X, Y, Z, T, X1 и адрес исполнения.
`poke`	Запись числа в X, Y, Z или T.
`1302`	Регистр R микросхемы К145ИК1302.
`ring`	Активное кольцо памяти M.
R…=	Запись числа в R0…RE, а в расширенном режиме и RF.
`kbd`	Нажатие физической клавиши по scan-code.
`cmd`	Нажатие клавиш, соответствующих opcode МК-61.
`reinit`	Чистая инициализация калькулятора без перезагрузки устройства.
`run`	Запуск программы или открытие сохраненного файла.
`disa`	Переключение дизассемблера на дисплее.

Сценарии, индикация и звук

Команда	Назначение
`open`	Открытие или запуск файла из хранилища.
`if`	Выполнение следующей команды по условию.
`print`	Вывод строки, байтовых escapes и регистров на текущий экран; прежде всего для M61.
`wait`	Неблокирующая пауза M61-сценария в миллисекундах.
`ret`	Возврат из M61-сценария или обработчика ловушки.
`bind`	Привязка введённого opcode или события кнопки down/up к обработчику M61.
`led`	Асинхронная последовательность состояний светодиода.
`beep`	Асинхронная последовательность звуков и пауз.

Хранилище

Команда	Назначение
`save`	Сохранение программы M61 в числовой файл или по пути, с подтверждением.
`load`	Загрузка программы M61 из числового файла или по пути.
`pwd`	Полный путь текущего каталога.
`cd`	Смена текущего каталога.
`ls`, `dir`	Просмотр каталога или одной записи.
`mkdir`	Создание каталога; `-p` создаёт весь путь.
`mv`	Перемещение или переименование файла/каталога.
`rm`, `del`	Удаление файла; `-r` удаляет дерево.
`rmdir`	Удаление пустого каталога.
`df`	Физическая ёмкость, квота узлов и размер FAT12-кластера.
`bench`	Read-only замер чтения файла или последовательного диапазона C6.
`fsget`, `fsput`	Машинная передача файлов для `tools/mkc.cmd`.
`format`	Форматирование всего хранилища C6 с подтверждением.
`fsls`, `fsrm`, `fsstat`	Синонимы `ls`, `rm`, `df`.
`fsclean`	Удаление записей нулевой длины.
`vlog`	Журнал трассировки виртуального FAT.

Сервис

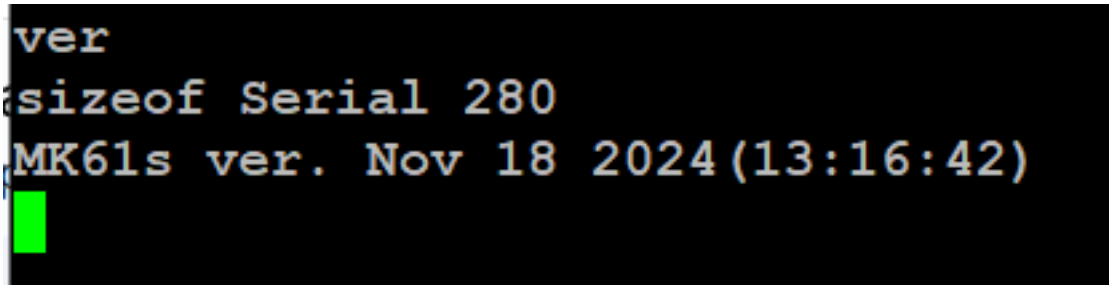
Команда	Назначение
`prof`	DWT-профиль времени ядра, дисплея, USB Screen, C6 и ZX0; снимок можно сохранить в текстовый файл.
`crash`	Просмотр, сохранение и очистка сохранённого аппаратного fault dump.
`wdog`	Состояние independent watchdog, причина последнего reset и retained breadcrumbs.
`mpu`	Состояние аппаратной защиты нулевого адреса, границы stack/heap и исполнения из SRAM.
`display`	Статус и стендовые тесты OLED1602/WS0010; в других профилях доступен только статус.
`uscreen`	Запуск ожидания desktop-клиента USB Screen, если возможность включена при сборке.
`rst`	Перезагрузка: подтверждение на устройстве либо явное `rst now`.
`dfu`	Немедленный переход в USB DFU-загрузчик.

Общие команды

`ver` — версия прошивки

```
ver
```

Команда печатает модель, дату и время сборки. Строка `sizeof Serial` — служебная диагностическая информация.



Пример вывода ver; версия на снимке может отличаться

`identity` — идентичность устройства и сборки

```
identity
```

Команда печатает одну версионированную машинно-читаемую строку: 64-битный public ID конкретного STM32, его короткий суффикс, фактическую USB serial строку STM32duino, отдельный FAT volume serial, build ID и профиль сборки. Полный заводской 96-битный UID наружу не выводится. Public ID стабилен после обновления прошивки и смены номера COM, но не является секретом или криптографическим ключом. `tools/mkc.cmd` использует эту строку, чтобы не перепутать MK61s с другой STM32 и различать несколько калькуляторов.

```
MK61 ID v=1 public=0123456789ABCDEF short=89ABCDEF usb=112233445566 volume=A1B2C3D4 build=12345678 profile=mini-v3-a00
```

`date` - дата и время

```
date
date ms
date 2026-07-19 14:35:00
```

Возможные ответы:

```
2026-07-19 14:35:07
2026-07-19 14:35:07.483
Set the date.
DATE OK
DATE ERROR: expected ms or YYYY-MM-DD HH:MM:SS
```

Без параметра команда печатает прежний seconds-only формат одной строкой; `date ms` добавляет согласованные доли секунды `.mmm`, не меняя совместимость старых хостовых программ. Если пользовательское время ещё не задавалось либо была потеряна резервная область RTC, следующей строкой выводится предупреждение `Set the date.` Параметр сразу задаёт новое значение: отдельное слово `set` не используется. Формат строгий, годы допустимы от 2000 до 2099, календарная дата проверяется с учётом високосных лет. Прошивка хранит введённое местное время и не пересчитывает часовой пояс. Полное описание приведено в документе `MK61s-mini-RTC.md`.

`alarm` — аппаратные будильники RTC

```
alarm
alarm a in 10
alarm b at 2026-08-30 07:15:00
alarm a daily 21:00:00
alarm b off
```

Без параметров выводятся retained-метаданные RTC и состояния двух независимых слотов A/B. `in` ставит точный относительно текущей субсекундной фазы разовый будильник через 1…31 536 000 секунд; `at` задаёт разовый календарный момент; `daily` повторяется ежедневно; `off` отменяет слот. Успех подтверждается `ALARM OK`, отказ проверки календаря, резервной записи или аппаратного включения — `ALARM ERROR`.

Срабатывание только подаёт короткий звук с учётом пользовательской громкости: оно не перехватывает экран и не запускает M61/APP. При нулевой громкости уведомление остаётся зарегистрированным, но беззвучно. Будильник способен разбудить питающийся STM32 из поддерживаемого режима сна, но не включает физически обесточенную плату. Команда запрещена из M61-сценариев и trap.

`help` — список команд

```
help
```

Список строится из того же реестра, по которому прошивка распознает команды, поэтому `help` — самый надежный способ проверить возможности установленной версии. В конце также выводятся специальные формы `R<r>=` и `set$`.

В F401/F411 ABI 6 описания хранятся в `/System/HELP0.TXT` и `HELP1.TXT`. Сборщик извлекает их из ELF той же прошивки, не занимая её Flash. Оба файла начинаются с подписи набора команд; устанавливайте их вместе со всем `/System` из одного комплекта. На F401 в него также входят `SETUP.APP` и `USBDISK.APP`. На F411 оба компонента resident по умолчанию; SETUP становится внешним при `MK61_SETUP_AS_APP=1`, а USB-диск — при `MK61_EXTERNALIZE_USBDISK=1`. Если справка отсутствует, устарела или C6 занят, `help` печатает список имён команд и синтаксис восстановления файлов через `fsput`. Команды терминала остаются доступными; второй копии полной справки в resident Flash нет ни на F401, ни на F411.

`history` — история ввода

```
history
```

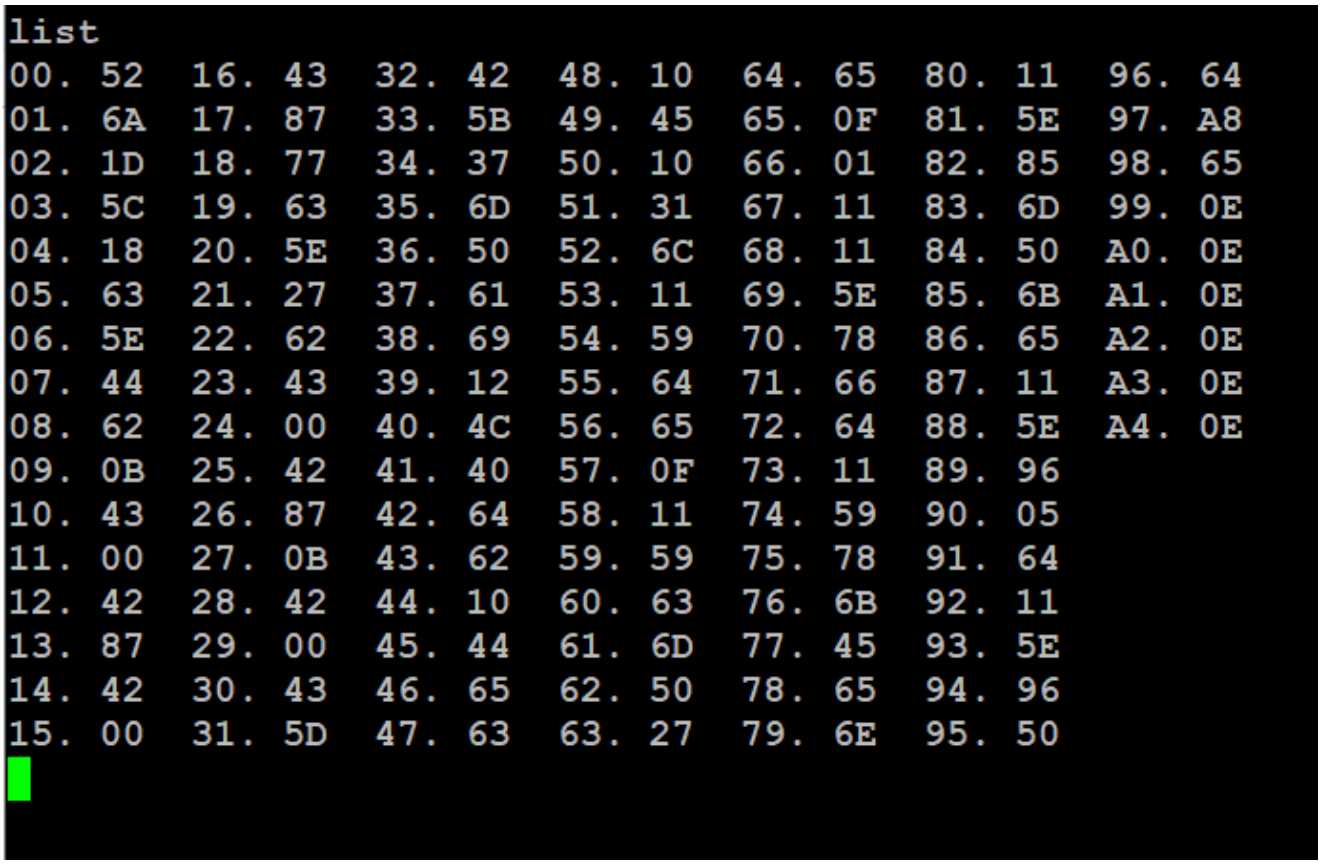
Команда нумерует строки от старой к новой. Стрелка вверх вызывает предыдущую строку, стрелка вниз возвращается к более новой или к строке, которую пользователь редактировал до входа в историю.

Программная память

`list` — вертикальная hex-таблица

```
list
```

Каждая ячейка выводится вместе с адресом шага программы.

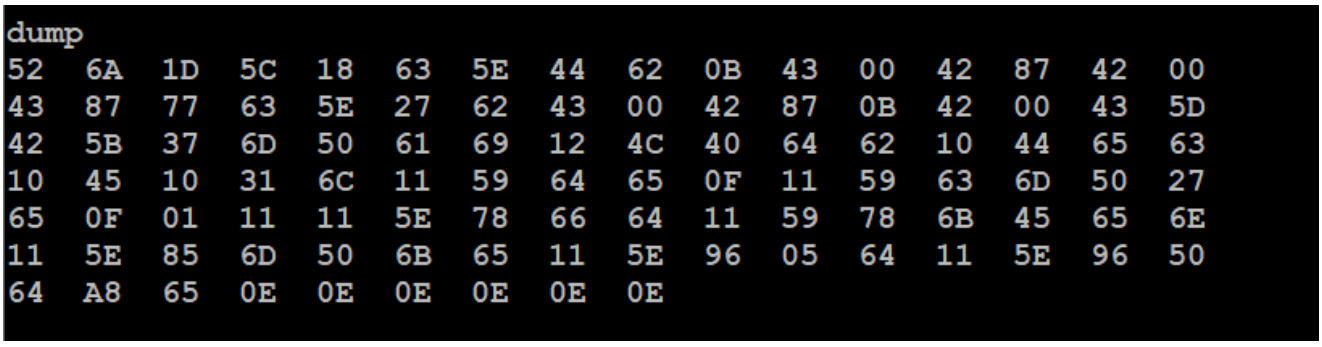


Пример вывода list

dump — непрерывный hex-листинг

```
dump
```

Удобен для быстрого копирования или сравнения последовательности opcode.

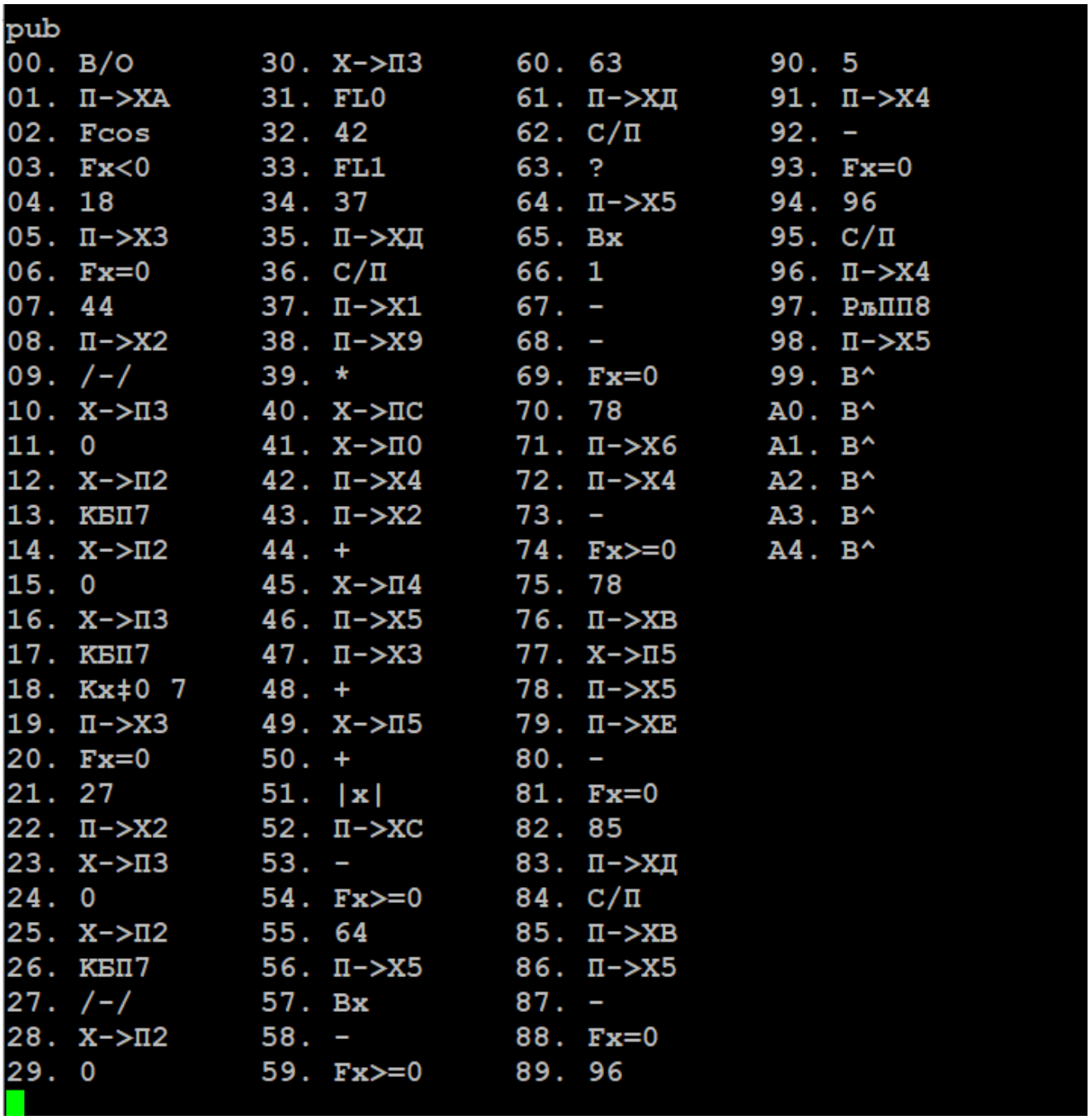


Пример вывода dump

pub — журнальный формат

```
pub
```

Печатает программу колонками в формате, близком к публикациям программ для советских программируемых калькуляторов.



Пример вывода pub

`lasm` — дизассемблированный листинг

```
lasm
```

Для каждого шага выводятся адрес, opcode и ассемблерная мнемоника. Второй байт двухбайтовой команды перехода показывается как ее аргумент.

Мнемоники английские: `add`, `sto3`, `ld3`, `hlt`. Листинг текущего банка содержит четыре колонки по 28 символов: шаги 00...29, 30...59, 60...89 и 90...B1. Адреса 100...109 обозначаются A0...A9, 110 и 111 - B0 и B1. Узкое окно терминала может переносить строки. Аргументы печатаются hex-байтами; их собственные строки содержат только адрес и байт, без ложной мнемоники.

В расширенном режиме `far` (`1F`) объединяет в строке opcode его подкод и адресные байты, а `disp` (`2F`) - подкод индикатора. Например, первые строки после `asm 0000 far jmp 02 50 disp 53 hlt`:

```
0000 1F far      51 02 50
0001 51
0002 02
0003 50
0004 2F disp     53
0005 53
0006 50 hlt
```

Для префикса на последних шагах аргументы читаются и из следующего банка; обычный локальный переход на последнем шаге берёт аргумент из начала текущего банка. В режиме 105 шагов 1F/2F не меняют разбор последующих байтов. `lasm` не меняет программу и не переключает банк. Без параметров он показывает текущую страницу 105/112 шагов; выбор диапазона этой командой не поддерживается.

`isa` — мнемоники ассемблера

```
isa
```

Печатает английские мнемоники в порядке opcode, в четыре колонки. Для байтов без отдельного имени показывается двухзначный hex-код: он тоже допустим в `asm`. Калькуляторные обозначения сохранены как синонимы ввода и используются в `pub`.

`asm` — ассемблерный ввод

```
asm [addr] <mnemonics>
asm 0000 1 2 add hlt
asm 3 4 mul
```

Необязательный `addr` — ровно четыре десятичные цифры начального адреса. Если адрес не указан, используется адрес трансляции, оставшийся после предыдущей успешной команды `asm`. Мнемоники разделяются пробелами; завершающий пробел не нужен.

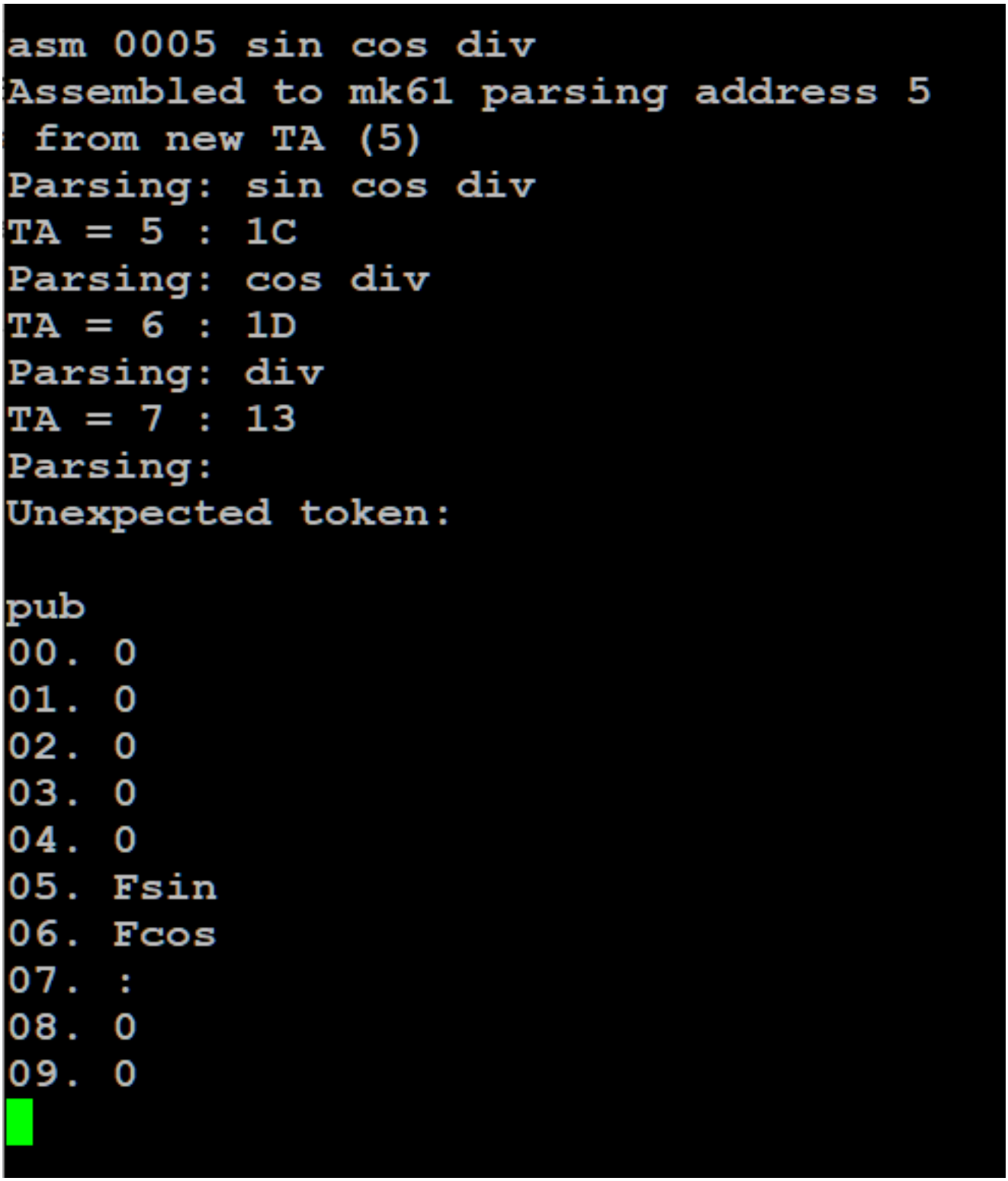
Можно смешивать английские имена, калькуляторные обозначения в CP1251 и двухзначные hex-байты. В калькуляторной мнемонике `Kx!=0 0` пробел отделяет регистр и является частью записи одной команды; вместо него допустимы несколько пробелов или табуляция. Регистр букв существенен: например, `stoA`, `ldF`, `preX`. `jlz` также принимается как синоним прямого `jl`.

```
asm 0000 1 2 + С/П
asm 0004 Kx!=0 0 jmp[A] 70
asm 0007 far jmp 02 50 disp 53 xmsr xmsp stoF ldF
```

Мнемоника	Байты	Назначение
`far`	1F	Префикс абсолютной адресации; подкод и аргументы следуют отдельно.
`disp`	2F	Префикс управления индикатором; далее один hex-подкод.
`xmsr` / `K1`	55	Обмен Ms с регистрами R0...R8.
`xmsp` / `K2`	56	Обмен Ms с программной памятью текущего банка.
`stoF`, `ldF`	4F, 6F	Запись и чтение RF.

Например, `far jmp 02 50` - байты `1F 51 02 50`, переход на абсолютный адрес 0250, а `far xmsp 18` - обмен с банком 24. Адресные байты `02 50` кодируются в BCD; номер банка `18` и подкоды `disp` - обычные hex-байты. Ассемблер записывает именно заданные байты и не вычисляет адреса по меткам. Семантика и допустимые последовательности приведены в описании расширений.

Строка сначала разбирается целиком и только затем записывается. Неизвестная мнемоника или выход за границу памяти оставляют прежнюю программу без частичной записи.

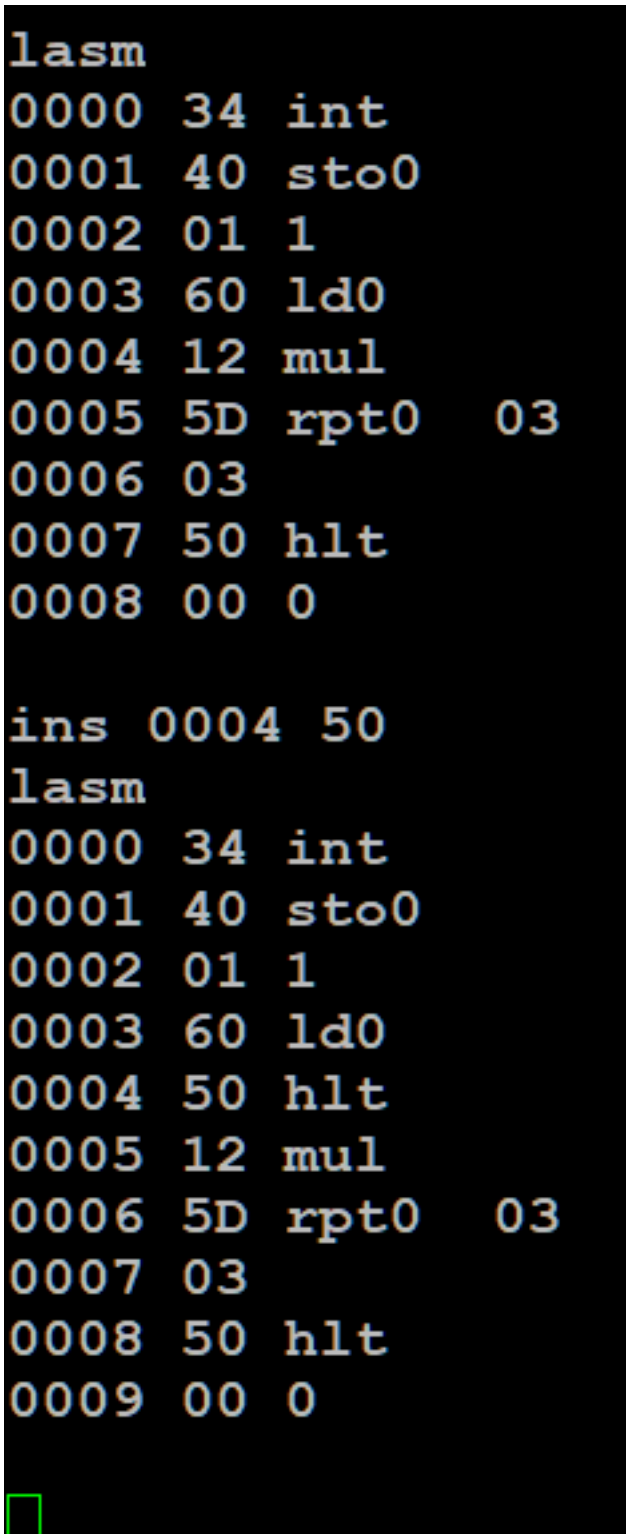


Пример ассемблерного ввода

`ins` — вставка opcode

```
ins <step> <opcode>
ins 10 20
```

`step` — десятичный номер шага, `opcode` — шестнадцатеричный байт `00`…`FF`. Команда вставляет байт, сдвигает последующую программу и корректирует адреса переходов.

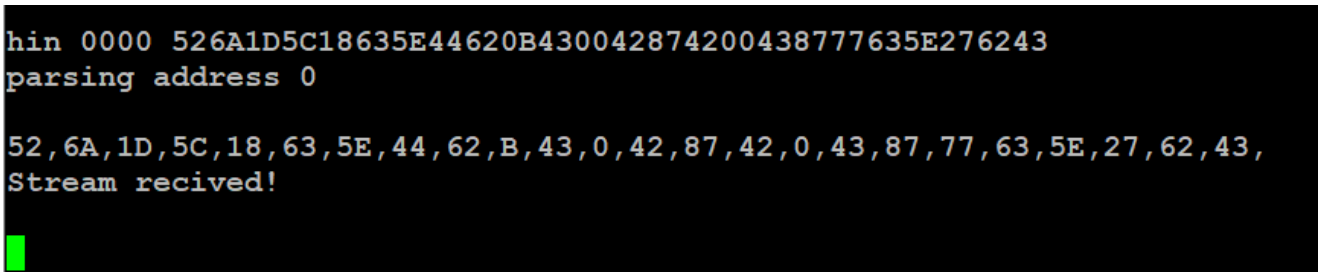


Пример вставки команды

`hin` — ввод hex-байтов

```
hin <addr> <hex-bytes>
hin 0000 010203040506
hin 0024 0E01030000000006
```

Адрес должен содержать ровно четыре десятичные цифры. Байты записываются слитно полными парами hex-цифр. Пробелы внутри строки байтов не допускаются. Запись за 105-й классический шаг автоматически включает расширенную программную память, если сборка ее поддерживает. В режимах `auto` и 112 шагов + RF адрес может быть абсолютным (`0000…9999`), включая дополнительные банки по 112 шагов. Команды адресации и индикатора описаны в расширениях 1F/2F.



Пример ввода hin

load <адрес> <путь.bin> — бинарная загрузка

```
load 0000 /games/ELITE/elite.bin
load 0224 "data block.bin"
```

Адрес — ровно четыре десятичные цифры (`0000`…`9999`). В терминале путь относителен текущему каталогу, в M61 — каталогу выполняемого файла. Один файл размещает байты подряд, автоматически переходя через границы 112-байтовых банков. Несколько вызовов могут заполнять разные диапазоны.

Формат `.bin`: CRC32/ISO-HDLC распакованных байтов, u32 little-endian, затем прямой поток ZX0 v2. CRC совместим с `zlib.crc32`. Размеров в заголовке нет: конец определяет маркер ZX0, после которого файл обязан закончиться. Максимальный размер файла — 4096 байт. C6 хранит его как тип `M3`, без перекодировки текста и без повторного сжатия.

Команда требует остановленного калькулятора. Она не очищает память и не запускает программу. В AUTO включается банковая память; принудительный режим 105 шагов ограничивает запись адресами `0000`…`0104`. Всего доступны 32 слота банков; нехватка слотов и выход за адресное пространство — ошибки.

При обрыве, неправильном потоке или CRC M61 останавливается до `run`. После неудавшейся распаковки `run`, `cmd`, `kbd` и изменение программы через `hin`/`asm`/`ins` блокируются. Уже записанные байты не откатываются. Выполните `reinit` и повторите образ либо заново откройте корневой M61. Ошибка открытия файла до распаковки не блокирует прежнюю программу.

Упаковщик из репозитория:

```
python3 tools/programs/program_pack.py raw-image.bin program.bin
```

Нужны Python 3 и C++17. Оптимальный ZX0-упаковщик собирается автоматически; Python запускает его и возвращает готовый бинарный файл. Вход содержит 1…3584 байта программы и данных, включая промежутки между банками. Сценарий задаёт адрес и запуск отдельно:

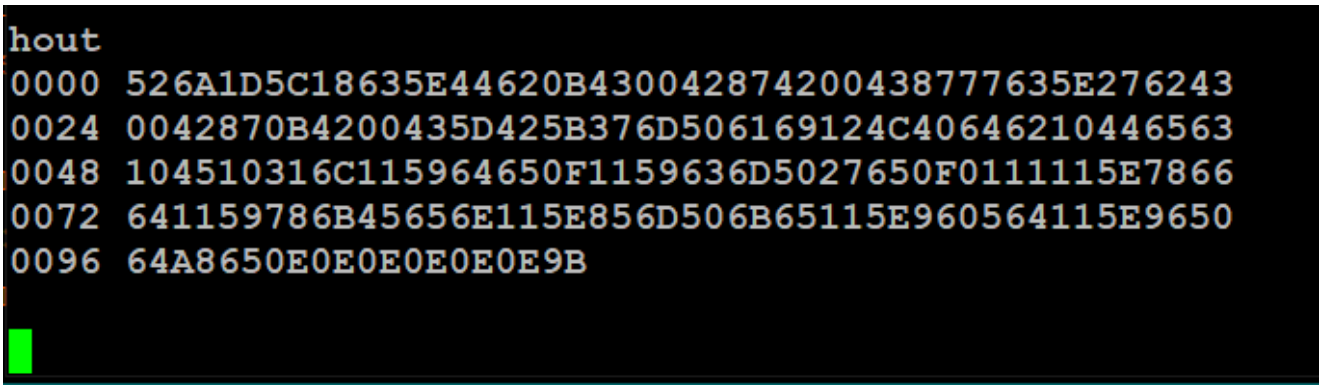
```
reinit
load 0000 program.bin
run
```

Команды `ztart`/`zin` удалены. load <номер>, load <путь.m61>, `hin` и `hout` сохраняют прежний формат. Числовые имена M61 с пробелом при необходимости можно заключить в кавычки целиком.

`hout` — вывод строк для `hin`

```
hout
hout 0250 24
```

Вывод разбит на строки с четырехзначным начальным адресом и группами до 24 байтов. Эти строки можно сохранить в текстовый файл или передать другому пользователю и затем выполнить как команды `hin`. Форма с адресом и числом байтов читает произвольный абсолютный диапазон. Без аргументов `hout` сохраняет прежний формат локальной страницы текущего банка.



Пример вывода hout

`clr` — очистка программы

```
clr
Y
```

После `clr` терминал ожидает отдельную строку `Y` или `y`. Отмена выполняется отдельной строкой `N` или `n`. Любая другая команда отменяет устаревший запрос подтверждения и затем обрабатывается как обычная команда.

`set$` — совместимая форма записи

```
set$0000 01020304
```

Форма выполняет ту же проверяемую hex-запись, что и `hin`, но адрес примыкает к имени. Для новых команд и файлов M61 предпочтительнее читаемая форма `hin`.

Стек, регистры и внутренняя память

`reg` — регистры R0…RE

```
reg
```

Показывает все классические регистры памяти в формате индикатора МК-61.

```
reg
R0 =  1.0000000  02
R1 =  1.0000000  01
R2 =  0.0000000  00
R3 =  0.1000000  00
R4 =  0.0000000  00
R5 =  6.0000000 -01
R6 = -0.7000000  01
R7 =  0.0000037  07
R8 =  0.0000099  07
R9 =  1.0000000  01
RA =  1.0000000  02
RB = -4.0000000  01
RC =  1.0000000  02
RD =  Г.ГГ0Г 00  00
RE = -3.9000000  01
IP:  99
```

Пример вывода reg

`stk` — стек

```
stk
```

Печатает X1, T, Z, Y, X и текущий адрес исполнения `IP`.

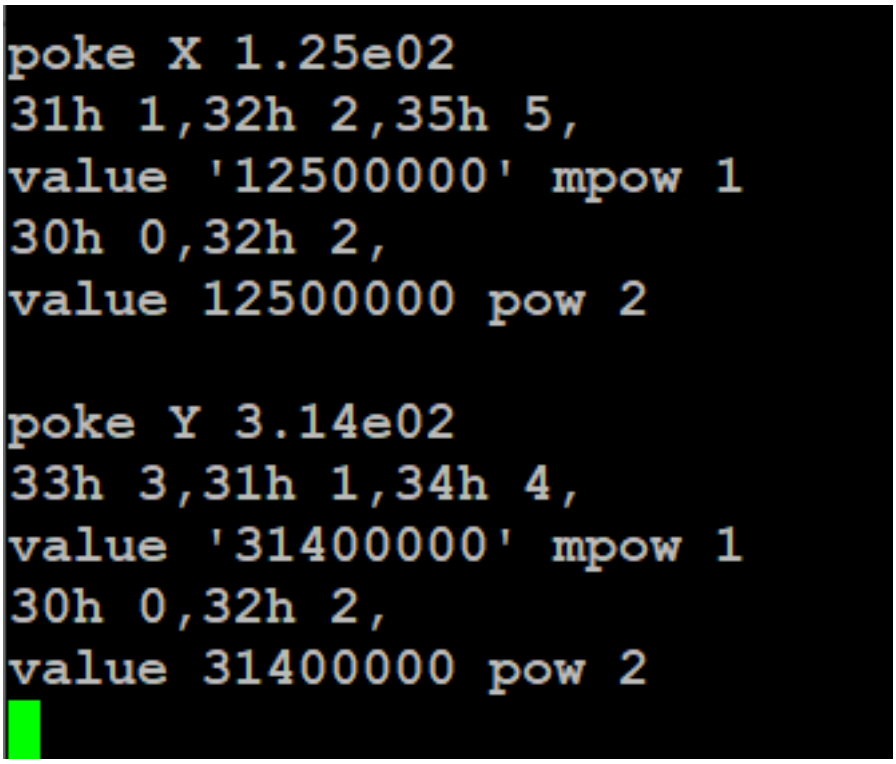
```
stk
X1 =  6.0000000 -01
T  =  0.0000000  00
Z  =  0.0000000  00
Y  =  0.0000000  00
X  =  0.0000000  00
IP =  99
```

Пример вывода stk

`poke` — запись в стековый регистр

```
poke X 1.25e02
poke Y -3.14
```

Допустимы X, Y, Z и T. Значение — конечное десятичное число, представимое форматом МК-61; порядок должен лежать в диапазоне от −99 до 99.



Пример записи poke

`R<r>=` — запись регистра памяти

```
R0= 3.14
RA= -1.25e02
RE= 0
RF= 42
R1= R0
R0= random
RB= raw 8CEC6ABA9907
```

Доступны R0…RE. RF разрешен только при включенной расширенной памяти. Значение проходит ту же проверку представимости, что и аргумент `poke`.

Имя регистра справа (`R1= R0`, `R0= ra`) копирует его числовое значение, сохраняя X и стек. Имена источника регистронезависимы, RF также требует расширенной памяти. Форма проверяет число и не копирует произвольные BCD-тетрады. Добавлена 29.10.2026.

`random` записывает ненулевую семизначную дробь `0.0000001`…`0.9999999` из отдельного аппаратно инициализированного потока. Форма не зависит от выбранного режима К СЧ и не сдвигает поток самой команды К СЧ.

`raw` предназначен для исторических программ, использующих регистр как 12-тетрадное хранилище. Следом указываются ровно 12 hex-тетрад в порядке: восемь тетрад мантиссы, её знак, знак порядка, десятки и единицы хранимого порядка. Вся строка проверяется до изменения регистра.

`1302` — регистр R К145ИК1302

```
1302
```

Это низкоуровневый диагностический вывод внутреннего регистра эмулируемой микросхемы.

```
1302
F08F00F08F08F08F08F08000F00F02F9FF9051C32F
```

Пример вывода 1302

`ring` — активное кольцо памяти M

```
ring
```

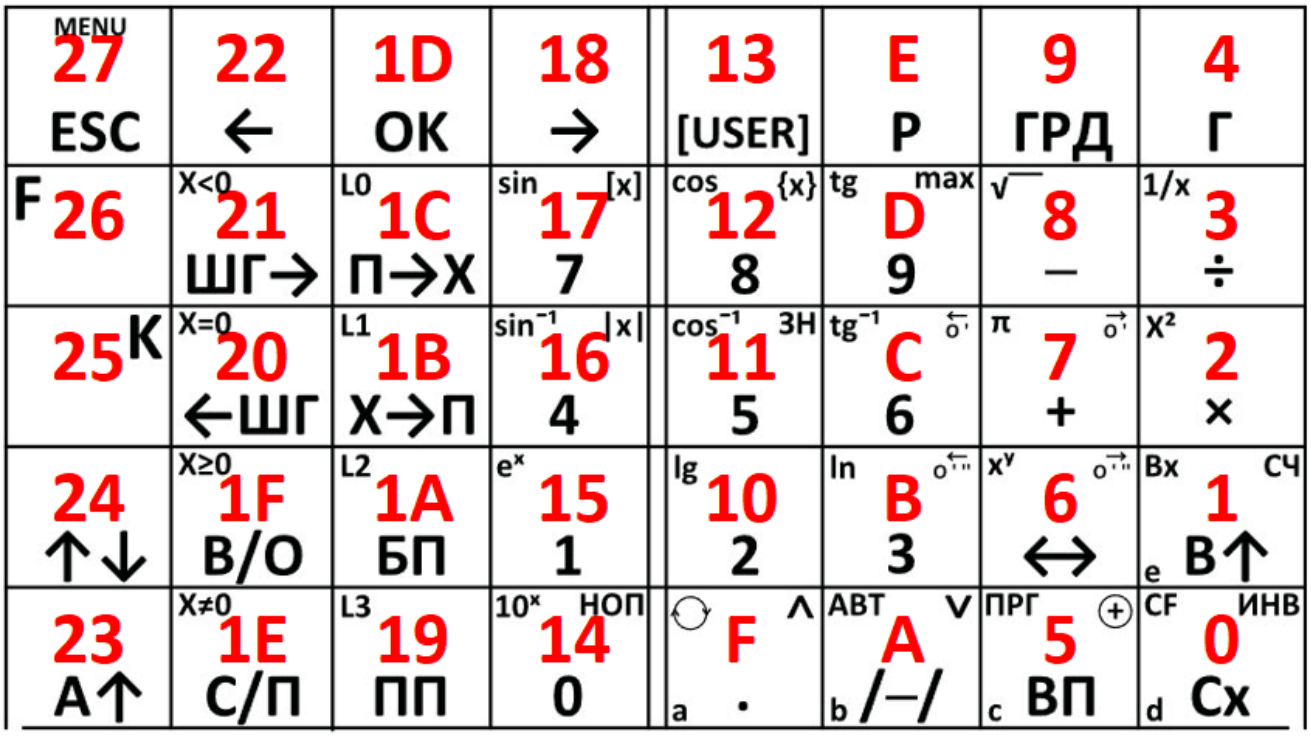
Печатает активные ячейки кольцевой памяти по эмулируемым микросхемам. Команда предназначена главным образом для диагностики ядра.

Клавиатура и запуск

`kbd` — физический scan-code

```
kbd <00..27>
kbd 02
```

Аргумент — шестнадцатеричный scan-code физической клавиатуры. Такая команда зависит от раскладки и нужна для точной имитации конкретной кнопки.



Scan-code клавиатуры

При входе через `kbd` в блокирующее сервисное меню терминал не сможет выполнять следующие команды, пока пользователь не выйдет из меню на самом устройстве.

`cmd` — операция по opcode

```
cmd <00..EF>
cmd 53
```

Прошивка преобразует opcode МК-61 в соответствующую последовательность клавиш. В отличие от `kbd`, эта форма не зависит от физического scan-code и лучше подходит для сценариев. Например, `cmd 53` соответствует операции ПП.

Старая команда `exec` больше не существует. Если нужен эквивалент нажатия ПП, используйте `cmd 53`.

`reinit` — чистое состояние калькулятора

```
reinit
```

Команда без подтверждения очищает программную память, стек, регистры, префиксы, ошибку, адрес исполнения и расширенное состояние МК-61. Текущий режим углов Р/Г/ГРД сохраняется. Активный М61-сценарий отменяется вместе с его ловушками `trap` и привязками `bind`.

Внутри обычного М61 команда продолжает тот же сценарий со следующей строки, но также снимает его активные `trap` и `bind`. В обработчике `trap` она запрещена. Не путайте её с `clr`, который очищает только программную память, и с `rst`, который перезагружает микроконтроллер и временно отключает USB.

Чтобы снять активные `trap` и `bind`, не очищая состояние калькулятора, удерживайте физическую клавишу `Cx`. Короткое нажатие `Cx` остаётся обычной операцией калькулятора.

`run` — запуск программы или файла

```
run
run 8
run DEMO
run DEMO.m61
```

Без аргумента команда имитирует последовательность `F`, `/-/`, В/О, С/П и запускает текущую программу с 00. Целый десятичный аргумент задаёт начальный адрес: `run 8` и `run 08` начинают с ячейки 08. Допустимы адреса 0…104 в режиме 105 шагов и 0…111 в расширенном режиме. Адрес относится к текущему банку; команда не загружает и не изменяет код программы.

С именем она работает как `open` и открывает файл из хранилища. Например, `run 8.m61` открывает файл. Для имени или номера файла, состоящего целиком из цифр, используйте `open 8`: числовой аргумент `run` всегда означает адрес. В сценарии М61 обе формы запуска программы ждут её останова, прежде чем перейти к следующей строке.

Только внутри файла М61 допустима форма перехода на метку:

```
run :loop
```

В интерактивном терминале эта форма возвращает ошибку.

`print` — вывод на экран из М61

```
print "TEST"
print "X={X}, X1={X1}, X2={X2}, RF={RF}\r\n"
print "\x1B[3;5H*"
print "\x1B[2K\rstatus"
print "14=\x14, 1B=\m1B"
print "\xFF"
```

Команда пишет в текущий экранный буфер, не очищает его и не добавляет перевод строки. Обычный текст заменяет только свои ячейки. Она понимает `\\`, `\"`, `\r`, `\n`, `\t`, `\b`, `\xHH`, `\mHH`, подстановки стека `X/Y/Z/T/X1/X2` и памяти `R0`...`RF` (также короткие `0`...`F`). `RF` требует расширенного режима. Суффикс `:m` выводит первую цифру нормализованной мантиссы, а `:e` — модуль десятичного порядка без ведущего нуля. Для `X2` эти форматы неприменимы.

Разряды регистров выводятся через ту же таблицу 16 состояний, что и индикатор МК-61:

Тетрада	Знак на экране
`0`...`9`	0, 1...9 (0 — экранный знак нуля)
A	-
B	L
C	C
D	Г
E	E

Тетрада	Знак на экране
F	пробел

Поэтому подстановка не теряет возникающие в разрядах L, C, Г, E, минус и пробел. {X}, {Y}, {Z}, {T}, {X1} и регистры памяти получают фиксированное регистровое представление. {X2} копирует непосредственно текущие 12 разрядов индикатора и положение запятой; именно её следует использовать для вывода снятого кадра программы МК-61.

Прошивка обрабатывает полезный монохромный поднабор ANSI: A/B/C/D/E/F, G, H/f, J/K, s/u и ESC 7/ESC 8; SGR не меняет монохромный экран. Координаты ограничены реальным размером активного дисплея. Обычные байты \xHH являются M8 и выводятся как соответствующие знаки: \x14 — круговая стрелка, \xFF — «я», а недопустимый \x00 заменяется вопросительным знаком. Старое \x1B продолжает начинать ANSI; для буквального надстрочного «ʸ» используйте \m1B или сам байт M8 в файле. В интерактивном терминале команда тоже доступна, но основной сценарий использования — .m61.

Первый успешный print из .m61 удерживает экран за сценарием до его завершения: промежуточная индикация работающего ядра подавляется, а выбранные кадры можно выводить из trap. Интерактивный print экран не захватывает.

wait <1..60000> доступна только внутри .m61 и асинхронно приостанавливает сценарий на указанное число миллисекунд. Внутри обработчика trap сохранённый контекст калькулятора остаётся приостановленным до ret.

Во время обычного wait при остановленном калькуляторе могут выполняться обработчики bind key. Их время входит в исходную паузу; собственное ожидание обработчика не переносит срок родительского. Привязки не прерывают друг друга и обработчики trap.

timer <0..60000|pause|resume> доступна в .m61 и управляет одним таймером текущего сценария. Число задаёт срок по монотонным часам в миллисекундах, 0 отменяет его, pause и resume сохраняют и продолжают остаток. В сценарных условиях if timer==0 ... проверяет истечение или отмену. Таймер учитывает время ПЗУ и ожиданий; реакция выполняется при следующей проверке условия. bind timer run :label поручает исполнителю M61 однократно вызвать обработчик по истечении срока, включая работу ПЗУ и wait. Вложенный файл имеет свой таймер, reinit очищает текущий. Долгое Cx снимает все привязки, включая таймер, и отменяет ещё ожидающее срабатывание. Примеры приведены в руководстве M61.

ret и trap — возврат и перехват M61

ret допустим только в .m61: он возвращает из локальной метки, обработчика trap или bind, из вложенного сценария либо завершает корневой сценарий. Если к этому моменту в корневом сценарии активированы ловушки или привязки, прошивка оставляет наблюдатель включённым. Несвязанный ESC снимает его. Объявление

```
trap 105 run :message
```

останавливает программу перед opcode шага 105 и передаёт управление метке. Краткое описание языка находится в руководстве M61 (MK61s-mini-M61.md), а полный путь ловушки по коду, правила сохранения контекста и ограничения обработчика — в документе «Команда trap в M61: семантика и реализация» (MK61s-mini-M61-Trap.md).

Привязка

```
bind AE run :message
```

перехватывает введённый с клавиатуры opcode AE, поглощает исходную операцию и вызывает метку. Команды программы с тем же opcode не перехватываются. Форма bind key 13 down run :help назначает сервисную клавишу ОК на экране калькулятора вне ПРГ. Она работает и во время run, приостанавливая шаги на время обработчика; в меню и просмотрщиках клавиша остаётся штатной. Форма bind key <код> down/up run :label поддерживает все 40 кнопок; коды и примеры приведены в руководстве M61. Обработчик завершается ret. Отдельно допускается до восьми перехватчиков opcode. Полные правила приведены в руководстве M61.

disa — дизассемблер на дисплее

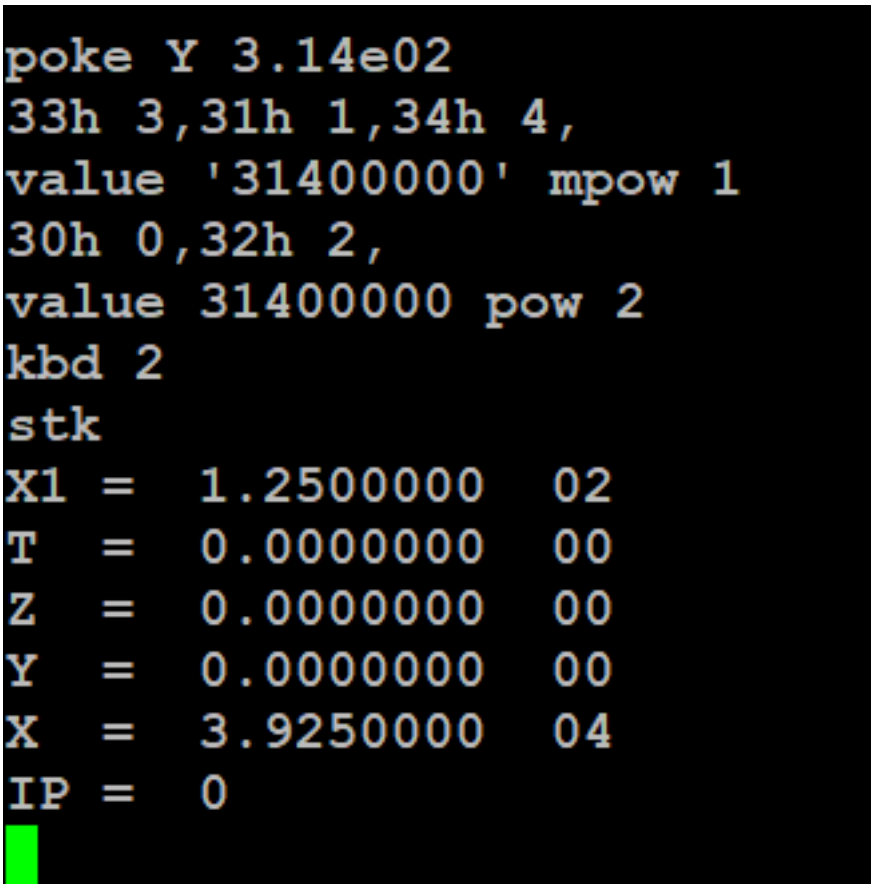
```
disa
```

Каждый вызов переключает режим верхней строки с дизассемблированной текущей операцией. Настройка действует не только в режиме ввода программы.

Пример последовательности действий

```
poke X 1.25e02
poke Y 3.14e02
kbd 02
stk
```

В X записывается 125, в Y — 314, затем нажимается клавиша умножения и проверяется результат в стеке. Конкретный scan-code следует сверять со схемой клавиатуры данной сборки.



Пример последовательности команд терминала

Условия, светодиод и звук

`if` — условное выполнение

```
if <left><op><right> <command>
if x<0 open NEGATIVE.txt
if r0>0 run :loop
if y!=0 beep 1200,80
```

Операнды могут быть числами, стековыми регистрами `x`, `y`, `z`, `t`, `x1` или регистрами памяти `r0`…`re`; `rf` доступен в расширенном режиме. Поддерживаются операции >, >=, <, <=, ==, `!=`. При ложном условии остаток строки не исполняется. При истинном условии он разбирается как обычная команда.

В интерактивном терминале полезны разовые условные действия. В M61 сочетание `if` и `run :label` образуют условные переходы и циклы. Внутри обработчика `trap` команда `run :label` является локальным вызовом: `ret` возвращает на следующую строку обработчика.

`led` — последовательность состояний светодиода

```
led 1
led 0
led 1,500,0,500,1
```

Нечетные элементы — состояния `0` или `1`, четные — длительности в миллисекундах. Последнее состояние задается без длительности. Допустимо до 16 состояний, каждая пауза — до 65535 мс. Паттерн выполняется асинхронно.

`beep` — звуковой паттерн

```
beep 4000,100
beep 4000,100,0,50,2000,200
```

Аргументы идут парами частота Гц, длительность мс. Частота `0` задает паузу. Допустимо до 16 пар; каждое значение — от 0 до 65535. Воспроизведение асинхронное и использует текущую настройку громкости.

Файлы и хранилище

Файловая система C6 поддерживает каталоги и типы файлов `M61`, `FOC`, `TBI`, `TXT`, `STATE.TXT`, `FMK`, `WBMP`, `CH8` и `APP`. Программы, тексты и шрифты ограничены 1536 байтами, исходники `.tbi` — 3584 байтами, изображения `.wbmp` — 1600 байтами, ROM `.ch8` — 3584 байтами, исполняемые контейнеры `.app` — 20 544 байтами. Basename занимает до 31 байта M8, глубина дерева — до 32 каталогов. Количество объектов вычисляется из реально измеренной ёмкости flash: например, для 512 КиБ доступно 192 узла, для штатной W25Q128 на 16 МиБ — 4084. Обычный файл и каталог занимают по узлу; большой APP, ROM и расширение большого каталога дополнительно расходуют служебные узлы. Точные значения установленного устройства показывает `df`.

Пути могут быть абсолютными (`/Programs/demo.m61`) или относительными текущему каталогу (`../Examples/demo.m61`). Принимаются `/` и `\`, а также компоненты `.` и `..`. Весь аргумент с пробелами заключайте в одинарные или двойные кавычки:

```
cd "/My Programs"
mv 'old name.m61' '../Archive/new name.m61'
```

Расширения и сравнение имён не зависят от регистра в FAT-проекции для ASCII и букв, представимых в M8/CP1251. Символы FAT `< > : " / \ | ? *`, управляющие байты, `./..`, завершающие пробел/точка и зарезервированные DOS-имена не допускаются. Если расширение при чтении опущено, терминал принимает basename только при единственном совпадении; при нескольких типах нужно указать расширение.

Загружаемые APP на F401

Отдельных терминальных команд для APP нет. Это обычные файлы C6, поэтому для них работают `ls`, `open`, `fsget`, `fsput`, `mv` и `rm`. Пользовательский APP может иметь любое допустимое имя и находиться в любом каталоге:

Команда	Пример для `.APP`
`ls /Apps`	Показывает APP вместе с остальными файлами C6.
`open /Apps/DEMO.APP`	Проверяет, динамически размещает и запускает приложение.
`fsget /Apps/DEMO.APP`	Скачивает контейнер без преобразований.
`fsput begin/data/end`	Потоково устанавливает или заменяет контейнер после полной проверки.
`mv /Apps/DEMO.APP /Archive/NEW.APP`	Перемещает или переименовывает как обычный файл.
`rm /Apps/DEMO.APP`	Удаляет APP и делает его блоки доступными сборщику мусора.
`df`	Учитывает inode, служебные extent и физические блоки в общей C6.

Системные компоненты имеют тот же формат. FOCAL, TinyBASIC, просмотрщики WBMP и Markdown и консоль CHIP-8 ищутся под каноническими путями `/System/FOCAL.APP`, `/System/BASIC.APP`, `/System/WBMP.APP`, `/System/MARKDOWN.APP` и `/System/CHIP8.APP`. На F401 добавляются `/System/SETUP.APP` и `/System/USBDISK.APP`. На F411 они требуются только при `MK61_SETUP_AS_APP=1` и `MK61_EXTERNALIZE_USBDISK=1` соответственно. `System` — обычный видимый каталог: его можно создать и копировать целиком через USB или MKC. Фиксированного числа APP нет: предел задают свободное место, общая квота inode и ёмкость каталога. Host-тест создаёт 200 APP; это не предельное число.

`open` — открыть файл

```
open DEMO.m61
open /Manual/README.txt
open /Pictures/SCHEME.wbmp
open /Games/PONG.ch8
open "../FOCAL examples/TEST.foc"
open /Apps/DEMO.app
```

Файл открывается или запускается в соответствии с типом. В M61-сценарии вложенный файл выполняется с последующим возвратом на следующую строку родительского сценария.

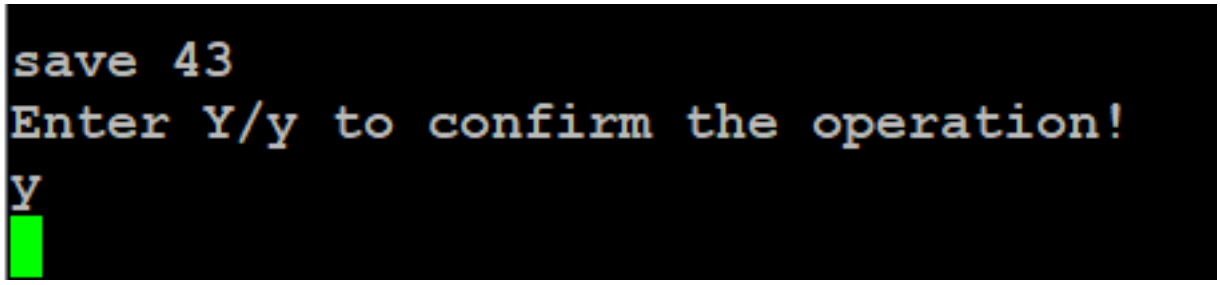
Вложенный M61 сохраняет уже загруженные банки; очистка программной памяти выполняется при новом корневом запуске из терминала или Проводника. Команда `reinit` позволяет явно очистить состояние внутри сценария.

Для `.wbmp` команда открывает тот же полноэкранный просмотрщик, что и Проводник; `.ch8` запускает обработчик консольного типа `C1`. Оба формата требуют текущий графический экран. На LCD1602 они доступны только при активном USB-экране, иначе команда сообщает ГРАФИКА / НЕДОСТУПНА. Пользовательский `.app` получает блок сверху свободной RAM, ниже защиты стека, и получает команду `APPLICATION_RUN`. `run <path>` является синонимом `open <path>`; `run` без аргумента запускает текущую программу. Подробности правил M61 приведены в `MK61s-mini-M61.md`, а разработка и сборка исполняемых файлов — в `MK61s-mini-APP.md`.

`save` — сохранить текущую программу

```
save 01
save /Programs/DEMO.m61
save "My Programs/DEMO 2"
Y
```

Принимается номер `0`…`99` либо абсолютный/относительный путь M61 с необязательным `.m61`. Чисто цифровой аргумент всегда означает корневой файл <номер>.m61, независимо от текущего каталога. Перед записью терминал ожидает отдельную строку `Y`/`y`; `N`/`n` отменяет операцию. Существующий M61 по тому же пути обновляется атомарно. Пустая программа не сохраняется.



Пример подтверждения save

`load` — загрузить M61

```
load 01
load ../Programs/DEMO.m61
```

В интерактивном терминале команда загружает программу в память. В M61-сценарии числовой или именованный M61-файл выполняется как вложенный сценарий и затем возвращает управление вызывающему файлу.

`pwd` и `cd` — текущий каталог

```
pwd
cd /Programs
cd ..
cd
```

`pwd` печатает абсолютный путь. `cd <path>` переходит в существующий каталог, а `cd` без аргумента возвращает в корень. Текущий путь отображается перед > в каждом приглашении терминала.

`ls` и `dir` — просмотр каталога

```
ls
ls /Programs
ls ../README.txt
dir
```

Без аргумента печатаются только непосредственные дети текущего каталога, без чтения и сортировки всего дерева. Для каталога выводятся строки `d`, для файла — `f`, размер в байтах и видимое имя с расширением. Аргументом может быть также один файл. APP перечисляются как обычные узлы C6. `dir` и `fsls` — синонимы `ls`.

`mkdir` — создать каталог

```
mkdir Programs
mkdir -p /Projects/FOCAL/Tests
```

Без `-p` родитель должен существовать. С `-p` недостающие компоненты создаются, а уже существующий путь считается успехом.

`mv` — переместить или переименовать

```
mv DEMO.m61 FINAL.m61
mv FINAL.m61 /Archive
mv /Projects/Old "/Projects/New name"
```

Если назначение — существующий каталог, исходное имя сохраняется. Иначе последний компонент задаёт новое имя. Тип файла изменять расширением нельзя, существующее назначение не перезаписывается, каталог нельзя переместить внутрь собственного поддерева или сделать дерево глубже 32 уровней. APP подчиняются обычным правилам.

`rm` и `rmdir` — удалить

```
rm DEMO.m61
rm -r /Projects/Obsolete
rmdir /Projects/Empty
```

`rm` удаляет файл немедленно. Для каталога требуется явный `rm -r`: дерево удаляется снизу вверх без рекурсивного расхода стека, после чего печатается число удалённых объектов. `rmdir` удаляет только пустой каталог. Корень удалить нельзя. Удаление APP атомарно убирает его из каталога; освободившиеся блоки позже возвращаются сборщиком мусора. `del` и `fsrm` — синонимы `rm` и также принимают `-r`.

`df` — ёмкость и квоты

```
df
```

Печатает измеренную физическую ёмкость SPI NOR, занятые/свободные/всего узлы, число видимых файлов и каталогов, служебные extents файлов и каталогов, виртуальный размер FAT12-кластера и объём сектора, зарезервированный под настройки. В конце без записи считываются JEDEC/SFDP/status внешней NOR и печатаются запрошенная частота SPI1, PCLK, аппаратный делитель и фактическая SCK. STM32 умеет выбирать только дискретные делители, поэтому запрос 20 МГц не гарантирует 20 МГц на выводе. В release-сборке следом выводится проверка целостности resident: `state`, обязательность, размер и смещение footer, совпадающие `expected/build/actual`, аппаратный или программный backend и число DWT-тактов. `not-run` означает, что метаданные footer не прошли проверку и CRC не запускался; это не следует принимать за успешную software-проверку. `fsstat` — синоним `df`. Строка `POWER` показывает аппаратный PVD, паспортные границы выбранной ступени, состояние write gate, события текущей загрузки,

breadcrumb предыдущей загрузки и число отклонённых program/erase/MSC-записей. `state=low` и `state=recovering` означают, что новые записи намеренно запрещены.

Строка `ANALOG` содержит время и число преобразований единого ADC-снимка, сырые коды VREFINT/temperature/VBAT, вычисленные VDDA, температуру кристалла MCU в десятых долях градуса и напряжение VBAT. `bat=unknown/no-detector` означает, что текущая плата не имеет отдельного детектора установленной батарейки: диагностическое напряжение плавающего входа нельзя считать доказательством наличия CR2032. В тесной production-сборке F401/UC1609 эта дублирующая диагностическая строка исключена ради обязательного запаса Flash; пользовательские показания в меню «Оборудование» остаются доступны.

`bench` — воспроизводимый замер чтения C6

```
bench file manual.md 5
bench flash 65536 3
bench random 1024 3
```

Первая форма читает существующий файл тем же API, которым пользуются просмотрщики, и подходит для сравнения времени открытия; размер ограничен 1536 байтами. Вторая читает без записи последовательный диапазон области данных C6 блоками по 512 байт. `random` тем же размером блока проходит псевдослучайные адреса с фиксированным seed, поэтому результаты разных прошивок сопоставимы. Для каждого прохода выводятся чистое время чтения и CRC-32, а затем min/avg/max и байт/с. Несовпавший CRC немедленно делает тест ошибочным. Команда ничего не создаёт, не стирает и не изменяет во внешней Flash. Она входит в квалификационные F411-сборки. В тесном production-профиле F401/UC1609 лабораторная команда по умолчанию исключена ради проверяемого запаса внутренней Flash; её можно явно вернуть флагом `MK61_ENABLE_READ_BENCHMARKS=1` для отдельного стендового образа.

`fsget` и `fsput` — машинная передача файлов

Эти команды являются транспортом двухпанельного менеджера `tools/mkc.cmd` и обычно не вводятся вручную. Они передают произвольный поддерживаемый файл C6, не меняя текущую программу калькулятора:

```
fsget "/Programs/demo.m61"
fsput begin "/Programs/demo.m61" 128 3141592653
fsput data 0 00112233AABBCCDD
fsput end
fsput cancel
```

По протоколу идут байты, которые реально хранятся в C6: M8 для текстовых типов и неизменённые байты для бинарных. `mkc` сам преобразует локальный UTF-8↔M8 по расширению; прошивка не угадывает кодировку и не меняет бинарные файлы.

`fsget` отвечает строками `@MKC:GET`, `@MKC:DATA` и `@MKC:END`. Загрузка начинается `fsput begin`, принимает последовательные HEX-блоки не более 96 байт и завершается `fsput end`; ответы — `@MKC:READY`, `@MKC:ACK` и `@MKC:DONE`. Ошибка имеет вид `@MKC:ERROR <code>`. Для APP код `PUT_FIRMWARE` означает несовпадение resident-прошивки, а `PUT_APP` — неверный заголовок, размер, сжатый поток или CRC.

Размер и контрольная сумма проверяются до атомарной записи C6. Контрольная сумма совпадает с POSIX `cksum`; в неё входит длина файла. Если между блоками рабочую память занял другой режим, загрузка отклоняется как прерванная. Любая команда, кроме следующего `fsput`, отменяет незавершённый сеанс. `fsget` и `fsput` намеренно запрещены внутри M61-сценариев.

Имя назначения обязательно содержит поддерживаемое расширение. Допустимы `.m61`, `.foc`, `.tbi`, `.txt`, `.state.txt`, `.fmk`, `.wbmp`, `.ch8`, `.app` и старые псевдонимы `.t1`, `.m2`, `.wbm`; действуют обычные ограничения C6 по имени и размеру. CHIP-8 ROM размером от 1 до 3584 байт и APP размером от 64 до 20 544 байт можно передавать по любому пути.

При `fsput` входной APP не хранится целиком в RAM. Прошивка принимает транспортные блоки последовательно, собирает их в одном 512-байтовом рабочем буфере и сохраняет под отдельными staging-key. После проверки POSIX `cksum` контейнер перечитывается потоково. Проверка заголовка и совместимости, CRC сохранённых данных, распаковки и CRC готового образа временно использует блок нужного размера, округлённого до 32 байт, сверху свободной RAM. Отдельного 20-КиБ резерва нет; это только лимит одной APP. Для ABI 6 выполняются ZX0, обратный BCJ при наличии флага и

проверка/применение релокаций. Загрузчик отклоняет ABI 2/3/4 до записи payload в RAM; старые приложения нужно пересобрать. Старый C6-файл в это время ещё не меняется. Только затем новые динамические блоки и дескриптор публикуются через COW/WAL. Ошибка оставляет прежний APP доступным; потеря питания даёт после перезапуска целиком старую либо целиком новую версию. Незавершённые staging-key удаляются при следующем сеансе или явном `fsput cancel`.

`format` — форматировать хранилище

```
format
Y
```

Команда переразмечает всю C6. Будут удалены все файлы и каталоги; журнал настроек имеет отдельную область и не затрагивается. Требуется отдельная строка `Y`/`y`; `N`/`n` отменяет операцию. Восстановить C6 можно и через пункт меню Система → Формат диска; Система → Полный сброс дополнительно удаляет настройки. Оба действия удаляют `/System` вместе с прочими файлами, поэтому во внешнем режиме `USBDISK.APP` затем потребуется установить заново через терминал. Это всегда нужно на F401 и нужно на F411 при `MK61_EXTERNALIZE_USBDISK=1`. В штатном F411 USB-диск встроен и доступен сразу после форматирования.

`fsclean` — удалить пустые записи

```
fsclean
```

Немедленно удаляет все файлы нулевой длины во всём дереве и печатает их количество. Полезно после оборванного или некорректного импорта через виртуальный USB-диск.

`vlog` — трассировка виртуального FAT

```
vlog
```

Выводит накопленный журнал USB/VFAT только в сборке с `MK61_VFAT_TRACE`. В обычной сборке сообщает, что трассировка пуста.

Сервисные команды

`prof` — аппаратный профиль времени

```
prof reset
prof start
prof stop
prof core 40
prof save PROFILE.TXT
prof
```

В поддерживаемой STM32-сборке команда использует счётчик тактов Cortex-M4 DWT CYCCNT. `start` включает сбор, `stop` останавливает её и печатает статистику, `reset` очищает все точки, а команда без аргумента печатает текущий снимок. Для каждой точки выводятся число измерений, минимум, среднее, максимум и сумма тактов. Частота ядра и измеренная цена самой пары чтений CYCCNT находятся в заголовке.

`prof core [1..1000]` запускает повторяемый тест ядра (по умолчанию ровно 40 полных `core.step`). Перед тестом прошивка сохраняет всё рабочее состояние калькулятора в уже существующий общий context-слот, временно создаёт известное чистое состояние ядра, а перед ответом восстанавливает и проверяет исходное; программа, стек и регистры пользователя не очищаются и не записываются во Flash. Команда отказывается работать во время RUN или когда context-слот занят M61/математическим backend.

Точки `flash.read`, `flash.write`, `flash.verify` и `flash.erase` раздельно показывают polling-чтение SPI NOR, page program, контрольное чтение после записи и стирание сектора. Поэтому end-to-end `fsput` может дать выборки одновременно в `flash.write` и `flash.verify`.

Строки `SPI1` и `SPI1 DMA` показывают состояние владельца общей шины, сбалансированность захватов и освобождений, число DMA-передач, байтов и входов в `WFI`, а также fallback, ошибки и timeout. В профилях mini длинные буферы от 64 байт передаются через DMA2 Stream 2/3 Channel 3; короткие команды и адреса остаются на polling-пути. Физические профили UC1609 пока целиком используют прежний polling-путь до отдельной проверки общего SPI1 на реальном экране.

Строка USB CDC RX показывает защиту входной очереди STM32 USB CDC. supported=1, linked=1 означает, что штатная сборка перехватила ошибочный путь Arduino Core с нулевым приёмным буфером: при заполнении очереди OUT endpoint отвечает NAK и возобновляется после чтения. throttles считает такие эпизоды backpressure и очищается командами prof start и prof reset.

Следующая строка CLASSIC относится к аппаратному регулятору скорости RUN: она показывает backend (TIM9 на STM32F401/F411), фактический период, состояние таймера, число тиков и исполненных шагов, отброшенные тики, текущую очередь и её максимум. В нормальном прогоне missed=0, а max_pending не превышает 1; очередь аппаратно ограничена двумя шагами, чтобы после долгой занятости не возникал неограниченный догоняющий рывок. prof start и prof reset очищают также эти диагностические счётчики.

В конце снимка повторяются строки POWER и, если она включена для данного профиля, ANALOG. ADC-снимок выполняется только при печати диагностики и не работает фоном, поэтому не добавляет периодических IRQ и не сокращает WFI idle.

Строки SLEEP описывают shallow Cortex-M WFI: число проверок и фактических входов, суммарное/минимальное/среднее/максимальное время сна, последнюю маску блокировки и счётчики причин отказа. Первая консервативная политика разрешает сон только в верхнем idle остановленного калькулятора. Его запрещают MSC, непосредственно ожидающая обработка terminal или USB Screen, звук, клавиша или debounce, CLASSIC, отложенное действие, вложенный viewer/menu и отсутствие SysTick wake. Сам по себе открытый CDC-порт или подключённый USB Screen больше не удерживает CPU активным: входящие USB-пакеты будят его прерыванием, а SysTick не позднее следующего миллисекундного тика запускает обычный scanner всех строк клавиатуры. STOP/STANDBY и изменение clock tree не используются. prof start и prof reset очищают также эти счётчики.

prof save [path.txt] останавливает сбор и записывает тот же снимок как текстовый файл C6; путь по умолчанию — PROFILE.TXT. Запись собственного отчёта не попадает в сохранённые счётчики flash.write, поэтому файл остаётся непротиворечивым и его можно скачать обычной командой fsget.

Пока сбор не запущен, каждая точка выполняет только быструю проверку флага. Размер-чувствительная сборка может полностью удалить профилировщик через -DMK61_ENABLE_DWT_PROFILER=0; тогда команда отсутствует в help. Сам WFI независимо выключается через -DMK61_ENABLE_IDLE_WFI=0.

crash — сохранённый аппаратный fault dump

```
crash
crash show
crash save [path.txt]
crash clear
```

На STM32F401/F411 обработчики HardFault, MemManage, BusFault и UsageFault записывают в 192-байтную область .noinit тип исключения, pc/lr, кадр регистров, MSP/PSP, маски Cortex-M4, CFSR/HFSR, адреса fault, reset flags, режим калькулятора и последние счётчики CLASSIC. Экспорт отчёта дополнительно содержит public ID платы и текущий профиль сборки, не меняя 192-байтный retained ABI. Запись защищена CRC и переживает программную перезагрузку; обработчик fault не обращается к USB, дисплею, SPI или файловой системе.

При следующем штатном запуске валидная запись автоматически сохраняется в C6 как один из четырёх кольцевых файлов CRASH0.txt…CRASH3.txt. Если C6 временно недоступна, SRAM-запись остаётся доступной. crash и crash show печатают полный отчёт, crash save записывает его по выбранному пути (по умолчанию CRASH.TXT), а crash clear явно удаляет только retained SRAM-record. Уже сохранённые файлы C6 эта команда не удаляет. Когда записи нет, команда показывает reset flags текущей загрузки и результат проверки .noinit layout.

Преднамеренные crash test ... существуют только в специальной стендовой сборке с MK61_ENABLE_FAULT_INJECTION=1; в release эти действия и их код отсутствуют.

wdog — independent watchdog

```
wdog
wdog status
```

На STM32F401/F411 команда показывает аппаратный backend IWDG, состояние драйвера, номинальный timeout, число завершённых проверок прогресса и фактических reload, максимальный интервал между ними

и отдельный признак `IWDGRSTF`. Watchdog запускается только в конце `setup()` и обычно кормится после обслуживания USB, terminal, звука, LED и дисплея. При длительном импорте USB-диска он также получает отметку после успешной физической записи C6 и после завершённого файла; обработчики прерываний его не кормят.

Текущий номинальный timeout — 32,768 секунды (`prescaler=256`, `reload=4095`). При максимальной частоте LSI минимальный интервал — около 22,3 секунды: это оставляет запас относительно измеренного на почти заполненном C6 интервала 18,8 секунды. После watchdog reset строка `WDOG previous` показывает последний валидный `.noinit` breadcrumb и отличает обычную работу от преднамеренного стендового зависания. При этом ложный аппаратный fault dump не создаётся: причина reset и exception — разные диагностические каналы.

Терминальная команда `dfu` при уже работающем IWDG сначала оставляет короткий retained handoff и выполняет software reset. Следующая ранняя загрузка входит в системный ROM bootloader до повторного запуска watchdog, поэтому длительная DFU-прошивка не прерывается.

Действия `wdog test starve` и `wdog test hang` существуют только в специальной стендовой сборке с `MK61_ENABLE_WATCHDOG_TEST=1`; в release их код и подсказка отсутствуют.

`mpu` — аппаратные границы памяти

```
mpu
mpu status
```

На STM32F401/F411 MPU запрещает доступ к нулевому адресу и ставит 256-байтную недоступную область между статическими данными/heap и зарезервированным стеком. Команда показывает фактические границы RAM, `_end`, guard, текущий и минимальный наблюдавшийся MSP и запас над guard. Это наблюдаемый минимум с момента старта, а не полный анализ high-water для каждой функции.

На F411 вся SRAM дополнительно помечена execute-never. На F401 исполнение из SRAM сохраняется, потому что загружаемые System APP действительно запускаются там; защита нулевого адреса и stack/heap guard при этом остаются. Если linker разместит статические данные в guard или аппаратных MPU-регионов недостаточно, защита не включится и `mpu` явно покажет `layout=invalid/enabled=0`.

Преднамеренные `mpu test guard|null|exec` доступны только в стендовой сборке с `MK61_ENABLE_MPU_TEST=1`. В release код этих действий отсутствует; реальные MemManage-проверки выполнены вместе с сохранением crash dump.

`display` — OLED1602/WS0010

```
display
display status
display reinit
display on
display off
display test text
display test alphabet 0
display test symbols
display test map 0
display test ddram 0
display test row 0 17
display test cgram
display test zero
display test cursor
display test clear
display test home
display test entry
display test autoshift
display test sleep
display test graphics 0
display test restore
```

`display status` сообщает контроллер, FT, карту DDRAM, состояние BF (`bf-seen`, `bf-timeouts`, `bf-fault`), текущий режим `character/graphics`, владельца графической транзакции `owner=none/api/qualification`, состояние OLED, маршрут USB Screen, тайм-аут сна, фазу запуска и число recovery. В профиле WEH001602A остальные подкоманды выполняют аппаратную приёмку: смешанный текст, алфавит, все 256 байтов CGROM, 2x64 DDRAM, независимую строку, CGRAM, cursor/blink, clear/home, entry shift и сон. Числа: `alphabet 0..2`, `map 0..7`, `ddram 0..63`, `row 0..1 0..63`.

`display test zero` скрыто записывает 256 нулевых байтов — существенно больше полевого сценария рассинхронизации — затем очищает DDRAM и должен показать `ZERO/BF 256: OK` / `4BIT SYNC: OK`. В терминале ожидаются `bf-seen=1` и `bf-timeouts=0`, `bf-fault=0`; `bf-seen=0` означает, что DB7/RW или уровни надо исследовать логическим анализатором, даже если фиксированный нижний предел пока даёт вывод.

Тесты временно заменяют содержимое физического экрана. После них выполните `display test restore`. У графического qualification-pattern есть также аппаратный выход: первое полное нажатие любой клавиши восстанавливает character mode, CGRAM/DDRAM и прежний интерфейс, но само это нажатие интерфейсу не передаётся. Пока `owner=qualification`, обычные символьные записи намеренно блокируются, поэтому терминал и фоновая перерисовка не могут превратить GDRAM в мусор. Графика доступна только в отдельной сборке с `MK61_WS0010_GRAPHICS_100X16=1` и не означает, что конкретный WEH001602A уже квалифицирован. Полный порядок приведён в `MK61s-mini-WS0010.md`.

`uscreen` — внешний USB-экран

```
uscreen
```

В сборке с `MK61_ENABLE_USB_SCREEN=1` команда переводит firmware в ожидание handshake с приложением MK61 USB Screen. Desktop-клиент отправляет её автоматически сразу после открытия CDC-порта, поэтому выбирать пункт меню на устройстве не требуется. Обычный terminal обслуживается из общего idle-loop, поэтому команда принимается, когда foreground-интерфейс был открыт с клавиатуры устройства. Если блокирующий viewer или редактор запущен самой командой `open`, текущий terminal-вызов ещё не завершён и следующая команда ждёт его возврата; в таком сценарии сначала выйдите из viewer физической клавишей. До успешного `ATTACH` физический экран остаётся включённым.

Команда не принимает аргументов и идемпотентна. В сборке с выключенным USB Screen она отсутствует в `help`. Полное описание handshake, приложения и выхода находится в `MK61s-mini-USB-Screen.md`.

`rst` — перезагрузка

```
rst
rst now
```

Подтверждение выполняется на дисплее и клавиатуре самого устройства. После подтверждения микроконтроллер перезагружается, а USB-порт временно исчезает. Форма `rst now` выполняет перезагрузку сразу и предназначена для автоматических аппаратных тестов; явное слово `now` защищает от случайного пустого Enter.

`dfu` — загрузчик DFU

```
dfu
```

Команда без дополнительного терминального подтверждения переводит устройство в режим USB Device Firmware Upgrade. Текущая терминальная сессия сразу заканчивается. Перед вводом убедитесь, что переход действительно нужен.

Команды в файлах M61

Строки сценария M61 используют тот же синтаксис, но выполняются с ограниченными правами. Разрешены:

- управление: `run`, `open`, `load`, `if`, `ret`, `trap`, `bind`;
- программа: `hin`, load <адрес> <путь.bin>, `set$`, `asm`, `ins`, `list`, `dump`, `pub`, `lasm`, `isa`, `hout`;
- калькулятор: `poke`, `R<r>=`, `reg`, `stk`, `1302`, `ring`, `kbd`, `cmd`, `reinit`;
- безопасные действия: `ver`, `print`, `wait`, `led`, `beep`.

Метки `:name`, `run :name`, `trap`, `bind` и `ret` существуют только внутри M61. Новая команда терминала автоматически не получает права сценария. В частности, `date`, `history`, `crash`, `wdog`, `uscreen`, команды удаления, форматирование, перезагрузка и DFU в M61 запрещены.

Полный синтаксис, вложенные файлы, метки и ограничения описаны в `MK61s-mini-M61.md`.

Подтверждения и осторожность

Терминальное подтверждение отдельной строкой `Y`/`N` используют:

- `clr`;
- `save`;
- `format`.

Обычный `rst` подтверждается на самом устройстве, а явный `rst now` — нет. `rm`/`del`/`fsrm`, в том числе рекурсивный `-r`, а также `mv`, `rmdir`, `fsclean` и `dfu` выполняются без терминального запроса. Для опасных команд проверяйте полный путь и тип файла до нажатия Enter.

Что изменилось относительно старой инструкции

- Команда часов `rtc [set ...]` заменена на `date [...]`; при неустановленном времени выводится предупреждение `Set the date.`
- Добавлены `help`, `history`, `ring`, `cmd`, `open`, `if`, `led`, `beep`, `bind`, `reinit`, именованные `save`/`load`, дерево C6 и команды `pwd`, `cd`, `ls`, `mkdir`, `mv`, `rm -r`, `rmdir`, `df`, `fsclean` и `vlog`.
- `run` теперь умеет открывать файл, а в M61 — переходить на метку.
- Первый аргумент `ins` — десятичный номер шага, второй — hex-opcode.
- `asm` больше не требует завершающего пробела и не изменяет память при ошибке разбора.
- Устаревшая команда `exec` удалена; эквивалент клавиши ПП — `cmd 53`.
- Подтверждение можно отменить отдельной строкой `N` или `n`.

Для установленной прошивки окончательным источником списка команд остается команда `help`.